**SASTRA DEEMED TO BE UNIVERSITY**
School of Computing — Department of Computer Science & Engineering

# PROJECT SYNOPSIS

## AUTOSAR AI – High-Level Design Document Intelligence Platform



| Student Name: | Hamdhan Ahmed B |
|---|---|
| **Register Number:** | 127003081 |
| **Degree & Branch:** | B.Tech. Computer Science & Engineering (AIML) |
| **GitHub Repository:** | https://github.com/hamdhan05/AUTOSAR-AI-Implementation |



## 1. Abstract

Modern automotive systems rely heavily on the AUTOSAR (AUTomotive Open System ARchitecture) standard to specify complex electronic control unit (ECU) software architectures. However, reviewing and verifying multi-hundred-page High-Level Design (HLD) specifications remains a manual, error-prone, and labor-intensive challenge. Furthermore, transmitting proprietary automotive intellectual property to public cloud AI models is strictly prohibited by automotive cybersecurity regulations (ISO/SAE 21434). This project introduces AUTOSAR AI, an on-premises document intelligence platform that automates the ingestion, structural entity extraction, architectural rule validation, and grounded conversational analysis of automotive HLDs. The system combines PyMuPDF for page-preserving extraction, SentenceTransformers (all-MiniLM-L6-v2) and ChromaDB for dense vector retrieval, in-memory BM25 for exact automotive identifier matching, and a local quantized Large Language Model (Ollama llama3.2:3b) for grounded question-answering with verifiable citations. Additionally, an independent Grounding Service decomposes generated responses into verifiable propositions to calculate a quantitative Grounding Score, and security defenses neutralize CSV formula injection attacks. The platform was validated through 28 automated pytest test cases (100% pass rate) and a 35-query benchmark evaluation.

## 2. Problem Statement

Automotive software engineering teams spend thousands of engineering hours manually cross-referencing architectural specifications across vehicle revision cycles. Generic document QA tools

frequently hallucinate technical component names, lack exact page citations, and cannot run in air-gapped, on-premises engineering environments. There is an acute need for a high-assurance, offline document intelligence system that combines grounded retrieval with automated architectural rule checking (e.g., detecting circular dependencies and insecure IPC bindings).

## 3. Project Objectives

- Develop an end-to-end PDF ingestion pipeline that preserves page numbering, headings, and character boundaries.
- Implement a hybrid retrieval pipeline combining dense vector embeddings with BM25 sparse keyword matching using Reciprocal Rank Fusion.
- Integrate a fully local Large Language Model (llama3.2:3b via Ollama) enforcing evidence-bounded prompt constraints and abstention.
- Build a claim decomposition and verification service that computes an empirical Grounding Score (0.0 to 1.0).
- Implement heuristic regex extractors to identify Software Components (SWCs), Runnables, Interfaces, Ports, and Signals.
- Detect architectural defects, including circular dependencies, insecure IPC communication, and missing port bindings.
- Provide an interactive 9-page Streamlit web user interface and 10 REST API endpoints in FastAPI with CSV export security.

## 4. Proposed System Architecture

The proposed platform is organized into eight decoupled layers: (1) Presentation Layer (Streamlit on port 8501); (2) API Gateway Layer (FastAPI on port 8000 with 10 routers); (3) Document Ingestion Layer (PyMuPDF); (4) Embedding & Vector Store (SentenceTransformers all-MiniLM-L6-v2 + persistent ChromaDB); (5) Hybrid Retrieval Layer (Dense + BM25 + Reciprocal Rank Fusion); (6) Local Inference Layer (Ollama on port 11434 with llama3.2:3b); (7) Relational Storage Layer (SQLite 3 with 14 SQLAlchemy ORM models); and (8) Reporting & Security Layer (CSV formula injection defense).

## 5. Technology Stack

| Subsystem | Technology Selected | Role & Justification |
|---|---|---|
| Frontend UI | Streamlit 1.32.0 | Reactive multi-page web application (9 pages) |
| Backend Framework | FastAPI 0.110.0 / Uvicorn | Asynchronous REST API with Pydantic type validation |

| | | |
|---|---|---|
| PDF Ingestion | PyMuPDF (fitz) 1.23.26 | High-speed text extraction with page provenance preservation |
| Vector Database | ChromaDB 0.4.24 | Local persistent vector storage and cosine similarity search |
| Embedding Model | all-MiniLM-L6-v2 (384-dim) | Lightweight, high-accuracy semantic embeddings |
| Local LLM Runtime | Ollama 0.40.3 (llama3.2:3b) | 100% on-premises, privacy-preserving generative inference |
| Relational Database | SQLite 3 / SQLAlchemy 2.0 | 14 relational tables with full foreign-key constraints |

## 6. Implemented Outcomes & Verification

All planned core modules have been fully implemented and verified. The platform features an automated test suite of 28 unit and integration tests executing across 12 test modules via Pytest 9.1.1, achieving a 100% pass rate in 27.35 seconds. The system successfully extracts SWCs, Runnables, Ports, and Signals; renders interactive Mermaid component topology graphs; generates grounded natural language answers with exact citations; flags rule violations; and exports sanitized CSV reports.

## 7. Limitations & Future Scope

Current limitations include reliance on text-searchable PDFs (raster scanned PDFs produce empty text) and heuristic regex extraction. Future work will incorporate: (1) Bidirectional ARXML AST parsing to generate formal AUTOSAR XML models directly; (2) Tesseract OCR for scanned legacy specifications; and (3) Real-time CAN/Ethernet bus timing simulation integration.